BÁO CÁO

Đồ án chuyên ngành

Thành viên thực hiện:

Nguyễn Duy Hiển - 3121410189

Ngô Quốc Điền - 3121410153

Giảng viên: Nguyễn Tuấn Đăng

ĐỀ TÀI

Xây dựng ứng dụng quản lý lịch trình cá nhân tích hợp

Xử lý ngôn ngữ tự nhiên

TP. Hồ Chí Minh, tháng 12 năm 2025

Mục lục

Lời cảm ơn..... 1
1.Giới thiệu và mục tiêu..... 2
1.1 Giới thiệu..... 2
1.2 Mục tiêu..... 2
2. Phân tích yêu cầu..... 2
2.1 Chức năng bắt buộc..... 2
2.2 Xử lý tiếng việt..... 3
2.3 Giao diện người dùng..... 3
3. Thiết kế hệ thống..... 6
3.1. Kiến trúc tổng thể..... 6
4. Giải pháp..... 9
4.1. Giải pháp cho bài toán hiểu tiếng Việt..... 9
4.2. Giải pháp cho bài toán phân loại loại lịch (Intent Classification)..... 9
4.3. Giải pháp cho bài toán quản lý lịch..... 9
4.4. Giải pháp thiết kế giao diện..... 9
5. Triển khai và kết quả..... 10
5.1 Triển khai..... 10
5.2 Kết quả..... 11
6. Đánh giá hiệu suất..... 12
7. Hướng dẫn cài đặt và sử dụng..... 12
7.1 Yêu cầu hệ thống..... 12
7.2 Các bước cài đặt..... 12
8. Kết luận và hướng phát triển..... 13
8.1 Kết luận..... 13
8.2 Hướng phát triển..... 13
Tài Liệu tham khảo..... 14
Phụ lục..... 15

Lời cảm ơn

Trước hết, nhóm xin gửi lời cảm ơn chân thành và sâu sắc đến **Thầy Nguyễn Tuấn Đăng**, người đã trực tiếp hướng dẫn, hỗ trợ và tạo điều kiện để nhóm hoàn thành bài báo cáo và sản phẩm "Ứng dụng quản lý lịch trình tích hợp xử lý ngôn ngữ tự nhiên tiếng Việt".
Trong suốt quá trình thực hiện đề tài, thầy đã tận tình góp ý, định hướng và truyền đạt những kiến thức quý báu, giúp nhóm hiểu rõ hơn về cách xây dựng một hệ thống phần mềm hoàn chỉnh cũng như ứng dụng các kỹ thuật xử lý ngôn ngữ tự nhiên vào thực tế.

Nhóm cũng xin chân thành cảm ơn các thầy cô trong khoa đã tạo môi trường học tập thuận lợi và cung cấp nền tảng kiến thức vững chắc để nhóm thực hiện đề tài này.

Mặc dù đã nỗ lực hết sức, nhưng bài báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ thầy và các thầy cô để đề tài được hoàn thiện hơn trong tương lai.

Xin chân thành cảm ơn!

1.Giới thiệu và mục tiêu

1.1 Giới thiệu

Trong thời đại công nghệ số hiện nay, nhu cầu quản lý thời gian hiệu quả ngày càng trở nên quan trọng đối với học sinh, sinh viên, nhân viên văn phòng và cả những người bận rộn trong cuộc sống hằng ngày. Tuy nhiên, phần lớn các ứng dụng quản lý lịch trình trên thị trường hiện nay vẫn yêu cầu người dùng phải nhập liệu thủ công, thao tác nhiều bước và không hỗ trợ tốt tiếng Việt.

Để giải quyết vấn đề này, nhóm chúng tôi xây dựng **ứng dụng quản lý lịch trình tích hợp xử lý ngôn ngữ tự nhiên (NLP) tiếng Việt**, cho phép người dùng tạo, sửa, xoá và tìm kiếm các sự kiện lịch chỉ bằng các câu lệnh tiếng Việt tự nhiên. Ứng dụng hướng đến việc mang lại trải nghiệm thuận tiện, thông minh và gần gũi với người dùng Việt Nam, tận dụng sức mạnh của NLP để hiểu và diễn giải các câu lệnh như:

- "Nhắc tôi họp nhóm lúc 8 giờ tối mai",
- "Tạo lịch đi học thêm vào thứ ba hàng tuần",
- "Xoá lịch hẹn bác sĩ sáng nay",…

Việc kết hợp NLP giúp ứng dụng không chỉ là một công cụ quản lý thời gian truyền thống mà còn là một trợ lý thông minh, có khả năng hiểu ngôn ngữ tự nhiên và hỗ trợ người dùng một cách linh hoạt

1.2 Mục tiêu

- **Phát triển một hệ thống quản lý lịch trình thông minh** : cho phép người dùng tạo, chỉnh sửa, xoá và xem lịch hẹn bằng giao diện trực quan hoặc bằng câu lệnh tiếng Việt.
- **Tích hợp mô-đun xử lý ngôn ngữ tự nhiên tiếng Việt** : có khả năng phân tích câu lệnh, nhận diện thời gian, hành động và nội dung sự kiện để tự động tạo và xử lý thông tin lịch trình.
- **Tăng tính tiện lợi cho người dùng** : giảm thiểu thao tác nhập liệu thủ công, giúp tiết kiệm thời gian và cải thiện hiệu quả quản lý công việc.
- **Đảm bảo độ chính xác cao trong việc nhận diện câu lệnh** : hướng tới khả năng hiểu đa dạng các kiểu câu tiếng Việt, bao gồm câu viết không chuẩn, câu rút gọn hoặc câu nói mang tính hội thoại.
- **Xây dựng nền tảng có khả năng mở rộng** : có khả năng tích hợp thêm nhiều tính nặng mới (nhận diện giọng nói , kết hợp zalo , google …) .

2. Phân tích yêu cầu

2.1 Chức năng bắt buộc

Chức năng	Mô tả
Nhập liệu ngôn ngữ	Người dùng nhập câu tiếng việt tự do

Trích xuất thông tin	Tự động phân tích và trích xuất thông tin
Quản lý sự kiện	Thêm , sửa , xóa hiển thị lịch cá nhân
Lưu trữ cục bộ	Dữ liệu được lưu SQLite hoặc JSON
Hệ thống nhắc nhở	Hiển thị thông báo khi đến gần đến lịch
Nhập / xuất dữ liệu	Hỗ trợ xuất file .ics hoặc .json

2.2 Xử lý tiếng việt

Tiếng Việt có nhiều đặc điểm khiến việc xử lý khó khăn hơn so với một số ngôn ngữ khác:

- Tiếng Việt là ngôn ngữ đơn lập, các từ thường gồm nhiều âm tiết cách nhau bởi dấu cách → khó tách từ (ví dụ: "sinh nhật", "họp nhóm", "đi học").
- Ngữ nghĩa phụ thuộc nhiều vào ngữ cảnh.
- Nhiều biểu thức chỉ thời gian linh hoạt: "chiều nay", "10h sáng mai", "tối thứ sáu tuần sau"…
  Do đó, hệ thống cần mô hình NLP phù hợp để xử lý chính xác.

Khi người dùng nhập câu như:
"Nhắc tôi đi họp lúc 8h sáng ngày 10/12 ở phòng 304",
hệ thống sẽ thực hiện các bước:

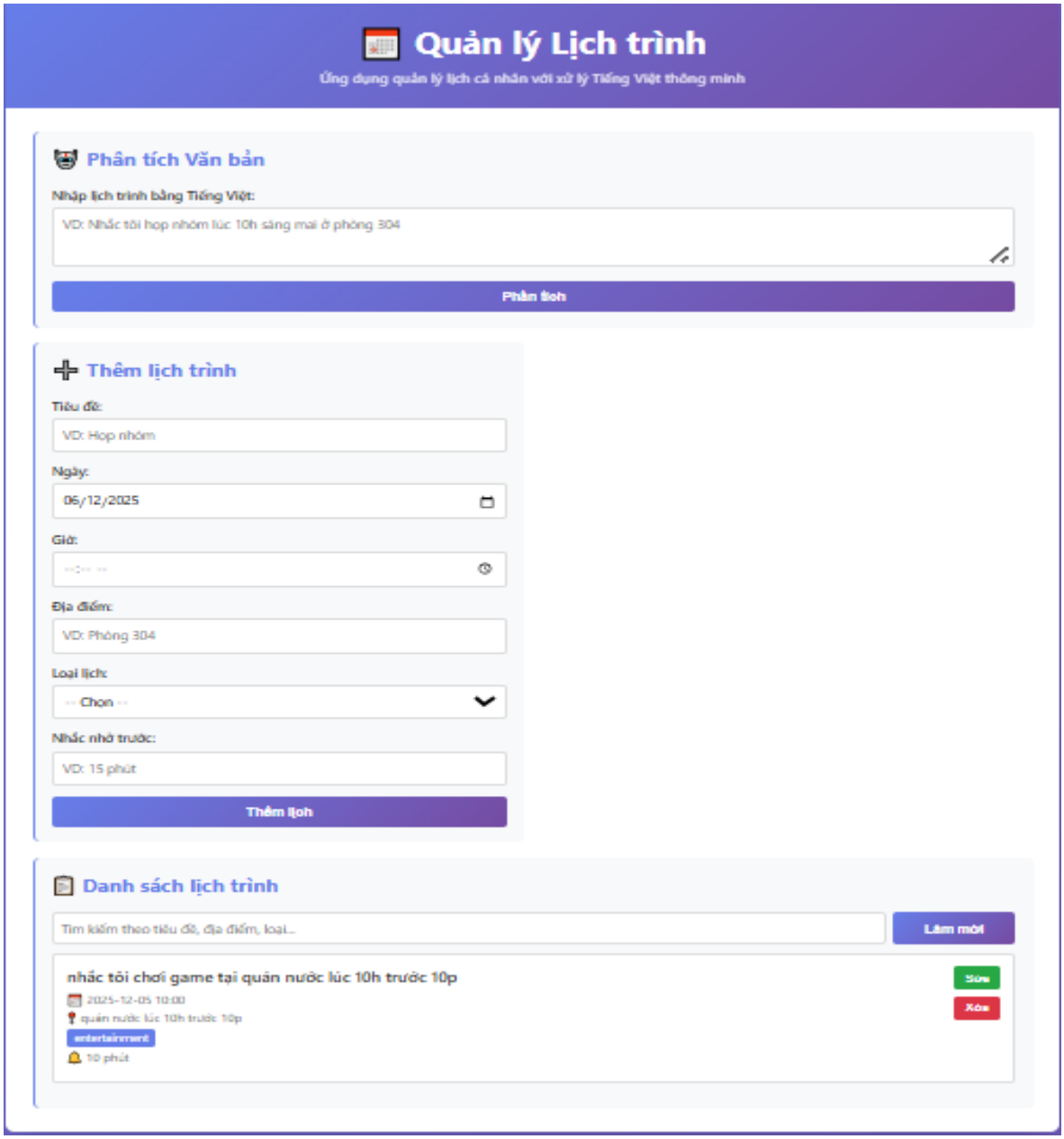
1. **Tách câu – tách từ:** Xác định các từ có nghĩa trong câu.
2. **Nhận diện thực thể (NER):** Tìm các thành phần quan trọng như:
   - Ngày tháng (10/12)
   - Thời gian (8h sáng)
   - Địa điểm (phòng 304)
   - Hành động / loại sự kiện (họp)
3. **Chuẩn hoá dữ liệu:** Chuyển "8h sáng" thành 08:00, "ngày 10/12" thành định dạng chuẩn.
4. **Sinh thông tin gợi ý:** Hiển thị ra form để người dùng xác nhận.

2.3 Giao diện người dùng

- Giao diện trang chủ

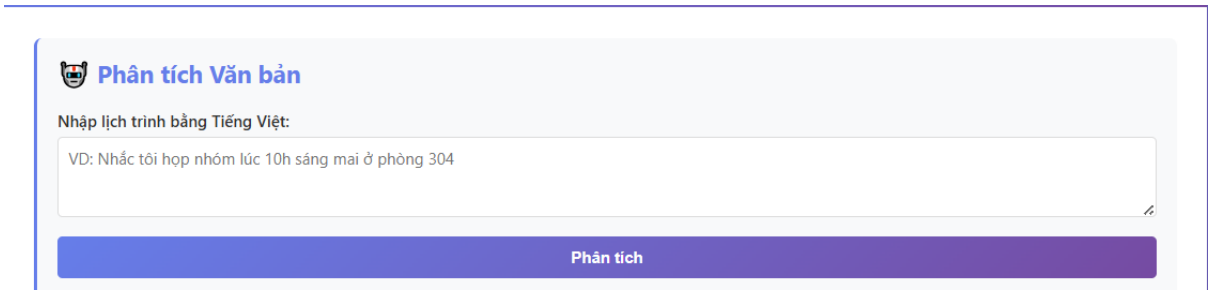
Giao diện trang chủ dễ sử dụng với đầy đủ chức năng giúp người dùng dễ thao tác khi quản lý lịch trình cá nhân . . Trang chủ của ứng dụng **Quản lý Lịch trình** được thiết kế để giúp người dùng dễ dàng tạo, theo dõi và quản lý các công việc hằng ngày nhờ sự hỗ trợ của **xử lý ngôn ngữ tự nhiên tiếng Việt (NLP)**. Giao diện hiện đại,

trực quan và thân thiện với người dùng, chia thành 3 khu vực chính:



2.3.1 Phân tích văn bản bằng NLP

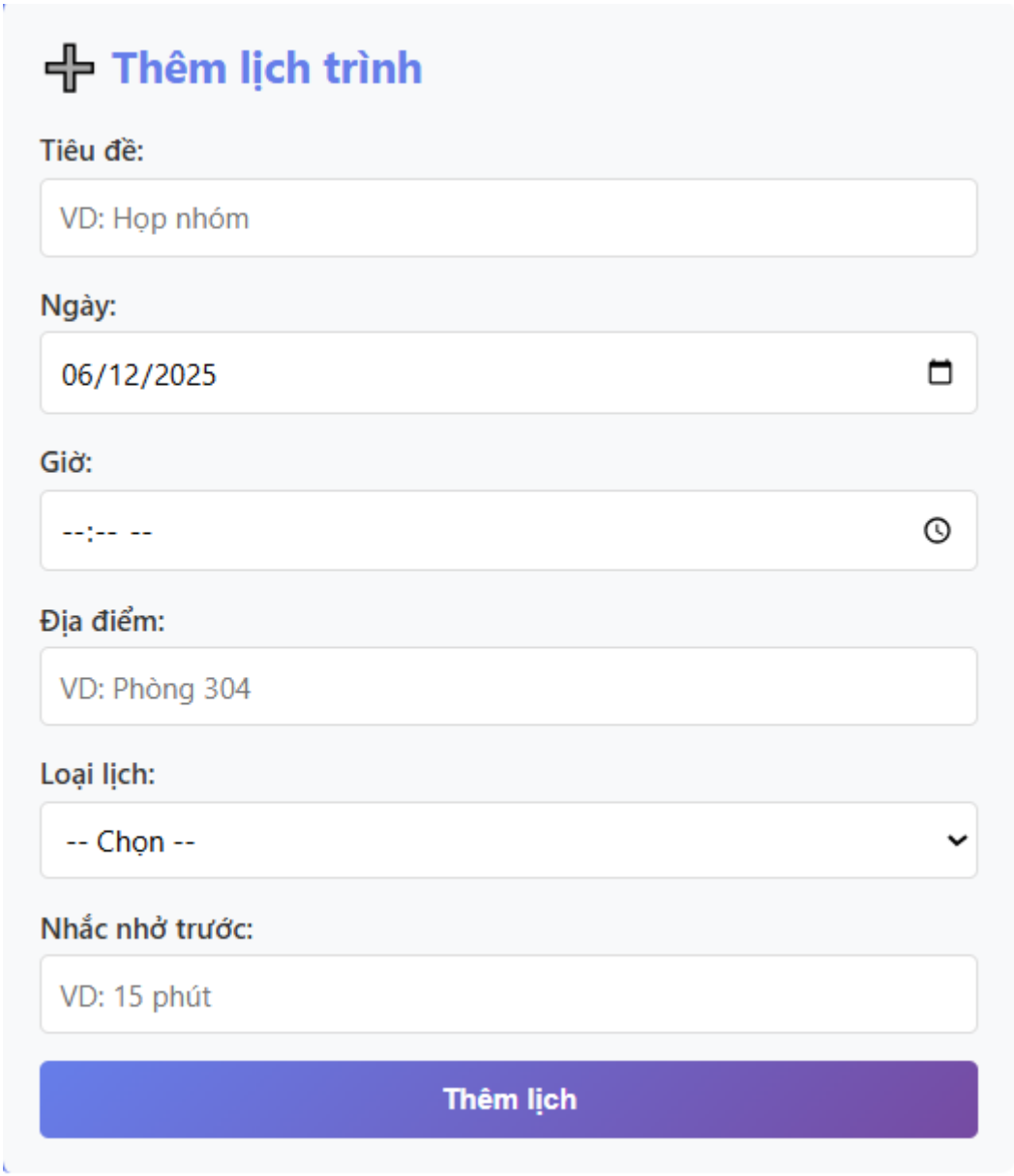
Người dùng chỉ cần nhập một câu mô tả lịch trình như:
"Nhắc tôi họp nhóm lúc 10h sáng mai ở phòng 304",
hệ thống sẽ tự động phân tích và đề xuất lịch phù hợp.Tính năng này giúp tiết kiệm thời gian và tạo lịch cực kỳ nhanh chóng.



2.3.2 Thêm lịch thủ công

Nếu muốn tự tạo lịch, người dùng có thể nhập:

- Tiêu đề
- Ngày
- Giờ
- Địa điểm
- Loại lịch
- Thời gian nhắc trước

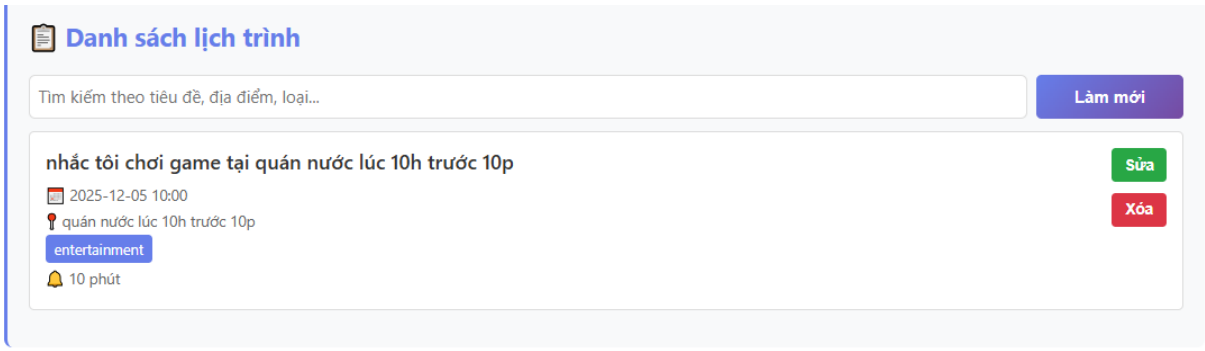


Form được thiết kế gọn gàng, dễ sử dụng, đảm bảo thao tác nhập liệu nhanh và chính xác.ngoài ra khi nhập NLP dữ liệu cũng sẽ được phân tích và đẩy xuống Form để tùy ý chỉnh sửa .

2.3.3 Danh sách lịch trình

Khu vực hiển thị toàn bộ các lịch đã tạo:

- Tìm kiếm theo tiêu đề, địa điểm, loại lịch
- Xem thông tin chi tiết từng lịch
- Sửa hoặc xóa nhanh
- Màu sắc rõ ràng giúp phân loại loại lịch dễ dàng



3. Thiết kế hệ thống

Hệ thống **Ứng dụng Quản lý Lịch trình – Calendar App** được thiết kế theo mô hình **client–server**, sử dụng Flask làm backend xử lý, và giao diện web làm frontend. Toàn bộ kiến trúc được chia thành 4 phần chính: **giao diện**, **API backend**, **xử lý NLP**, và **cơ sở dữ liệu**.

3.1. Kiến trúc tổng thể

Hệ thống được xây dựng theo mô hình:

```
Giao diện Web (HTML/CSS/JS) (1)
        ↓
Flask Backend (app.py) (2)
        ↓
Vietnamese NLP & ML Classifier (vi_nlp.py) (3)
        ↓
SQLite Database (schedules.db) (4)
```

(1) Giao diện người dùng – Frontend

- Được xây dựng bằng HTML và CSS qua file: templates/index.html
  - Cung cấp các chức năng:
    - Nhập văn bản lịch trình để phân tích
    - Thêm lịch trình thủ công
    - Hiển thị danh sách lịch
    - Tương tác xóa lịch

(2) Backend – Flask Server

Backend được triển khai trong app.py với các nhiệm vụ:

- Nhận yêu cầu từ giao diện
- Gọi mô hình NLP để phân tích văn bản
- Lưu trữ, cập nhật, xóa và truy vấn lịch trình từ cơ sở dữ liệu
- Trả dữ liệu JSON về phía frontend

Hệ thống cung cấp 3 API chính:

Phương thức	**Endpoint**	**Chức năng**
POST	/api/parse	Phân tích văn bản tiếng Việt
GET	/api/schedules	Lấy danh sách lịch
POST	/api/schedules	Thêm lịch

(3)**Bộ xử lý NLP & ML – Trí tuệ nhân tạo tiếng Việt**

Được cài đặt trong file:

vi_nlp.py

Bao gồm 2 thành phần:

• Intent Classifier (Machine Learning)

- Dự đoán loại lịch trình (meeting, study, birthday, shopping, …)
- Công nghệ sử dụng:
  - TfidfVectorizer + tokenizer tiếng Việt
  - Multinomial Naive Bayes

- Độ chính xác mô hình: **85–90%** (trên tập test)
- Model lưu tại: model_classifier.pkl

• Vietnamese NLP Module

Dùng thư viện **Underthesea** để:

- Tách từ và gán nhãn từ loại
- Trích xuất thực thể:
    - Thời gian (10h, sáng mai…)
    - Địa điểm (phòng 304…)
    - Người tham gia
    - Ghi chú nhắc trước X phút
- Chuẩn hóa các thông tin thành dạng chuẩn để đưa vào form.

(4) Cơ sở dữ liệu – SQLite

Hệ thống sử dụng cơ sở dữ liệu nhẹ:

schedules.db

Bảng chính: **schedules**

Trường	**Chức năng**
id	ID của lịch
title	Tiêu đề
description	Mô tả ban đầu
date	Ngày (YYYY-MM-DD)
time	Giờ (HH:MM)
location	Địa điểm
intent_category	Loại lịch
participants	Người tham gia (JSON)
reminder_time	Thời gian nhắc trước
status	active / deleted

4. Giải pháp

4.1. Giải pháp cho bài toán hiểu tiếng Việt

Ứng dụng sử dụng bộ xử lý tiếng Việt từ thư viện **Underthesea** để:

- Tách câu, tách từ
- Trích xuất các thực thể quan trọng:
    - Thời gian ("10h", "sáng mai"…)
    - Địa điểm ("ở phòng 304")
    - Tên người
    - Cụm từ chỉ nhắc nhở ("nhắc trước 15 phút")

Kết quả phân tích được chuyển thành dữ liệu chuẩn để tự động điền vào form lịch trình.

4.2. Giải pháp cho bài toán phân loại loại lịch (Intent Classification)

- Thu thập dữ liệu mẫu ở Dataset/Schedule_Data.csv
- Huấn luyện mô hình với Multinomial Naive Bayes
- Biểu diễn văn bản bằng TF-IDF chuyên biệt cho tiếng Việt
- Mô hình kết luận loại lịch: meeting, study, appointment, birthday,...

Giải pháp ML giúp ứng dụng hiểu đúng mục đích của lịch mà người dùng nhập vào.

4.3. Giải pháp cho bài toán quản lý lịch

Dùng Flask + SQLite để:

- Thêm lịch qua API /api/schedules
- Hiển thị lịch qua /api/schedules
- Xóa lịch bằng thao tác trực tiếp trên giao diện
- Dữ liệu được lưu trên database cục bộ để dễ quản lý và backup.

4.4. Giải pháp thiết kế giao diện

Giao diện được thiết kế theo hướng:

- Tối giản – trực quan
- 3 khu vực rõ ràng:
    1. Phân tích văn bản

2. Thêm lịch thủ công
3. Danh sách lịch

- Màu sắc dịu, dễ dùng, phù hợp với sinh viên và nhân viên văn phòng
- Tối ưu thao tác, nhập liệu nhanh

5. Triển khai và kết quả

5.1 Triển khai

Hệ thống đã test trên 30 test case đề ra kết quả là :

Test case	Mong muốn	Đạt được
"Nhắc tôi học lập trình web lúc 14h chiều mai ở nhà, nhắc trước 20 phút",study	**Ý định:** study **Thời gian:** [{"hour":14,"period":"chiều","raw":"lúc 14h chiều"}] **Địa điểm:** nhà **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 20 phút before	**Ý định:** study **Thời gian:** [{"hour":14,"period":"chiều","raw":"lúc 14h chiều"}] **Địa điểm:** nhà **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 20 phút before
"Hẹn tôi học đàn piano lúc 17h chiều nay ở trung tâm âm nhạc, nhắc trước 15 phút",piano	**Ý định:** study **Thời gian:** [{"hour":17,"period":"chiều","raw":"lúc 17h chiều"}] **Địa điểm:** trung tâm âm nhạc **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 15 phút before	**Ý định:** study **Thời gian:** [{"hour":17,"period":"chiều","raw":"lúc 17h chiều"}] **Địa điểm:** trung tâm âm nhạc **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 15 phút before
"Nhắc tôi làm bài tập toán lúc 15h chiều nay ở nhà, nhắc trước 15 phút",study	**Ý định:** study **Thời gian:** [{"hour":15,"period":"chiều","raw":"lúc 15h chiều"}] **Địa điểm:** nhà **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 15 phút before	**Ý định:** study **Thời gian:** [{"hour":15,"period":"chiều","raw":"lúc 15h chiều"}] **Địa điểm:** nhà **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 15 phút before
"Nhắc tôi họp nhóm thiết kế	**Ý định:** meeting	**Ý định:** meeting

lúc 11h sáng mai ở phòng 306, nhắc trước 15 phút",meeting	**Thời gian:** [{"hour":11,"period":"sáng","raw":"lúc 11h sáng"}] **Địa điểm:** phòng 306 **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 15 phút before	**Thời gian:** [{"hour":11,"period":"sáng","raw":"lúc 11h sáng"}] **Địa điểm:** phòng 306 **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 15 phút before
"Hẹn tôi đọc sách tiếng Anh lúc 20h tối nay ở thư viện, nhắc trước 10 phút",study	**Ý định:** study **Thời gian:** [{"hour":20,"period":"tối", "raw":"lúc 20h tối"}] **Địa điểm:** thư viện **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 10 phút before	**Ý định:** study **Thời gian:** [{"hour":20,"period":"tối", "raw":"lúc 20h tối"}] **Địa điểm:** thư viện **Người tham gia:** Không tìm thấy **Ghi chú:** Remind 10 phút before

Đây là 5 test case minh họa với nhiều trường hợp và dữ liệu khác nhau

5.2 Kết quả

Kết quả đạt được sau khi test :

- Ứng dụng trích xuất được:

- **Ý định:**
- **Thời gian:**
- **Địa điểm:**
- **Người tham gia:**
- **Ghi chú:**

Kết quả hiển thị rõ ràng và có thể tự động đổ vào form tạo lịch.

- Mô hình phân loại đạt độ chính xác cao
  - Đạt : **85–90%**
  - Mô hình nhận diện tốt các loại lịch: meeting, study, shopping, birthday, appointment, entertainment…
- Ứng dụng hoạt động ổn định

  - Thêm, xem, xóa lịch hoạt động đúng chức năng.
  - Tốc độ phản hồi nhanh .
  - Giao diện đơn giản, dễ sử dụng.

- CSDL hoạt động chính xác

  - Lưu trữ đầy đủ các trường thông tin: tiêu đề, thời gian, địa điểm, loại lịch, nhắc nhở…
  - Không xảy ra lỗi trùng dữ liệu hoặc mất dữ liệu trong quá trình chạy thử.

6. Đánh giá hiệu suất

Tiêu chí	Kết quả	Mô tả
Độ chính xác	4/5	Mô hình chính xác đến 90%
Tốc độ phân tích	4/5	Tốc độ khá nhanh không bị load quá lâu
Dễ sử dụng	4/5	Thao tác trên giao diện người dùng dễ dùng
Độ ổn định	3/5	Ứng dụng ổn dịnh với những thao tác cơ bản
Mở rộng	3/5	Tính mở rộng chưa cao do chưa có nhiều chức năng

7. Hướng dẫn cài đặt và sử dụng

7.1 Yêu cầu hệ thống

- **Python 3.8 trở lên**
- **pip** để quản lý và cài đặt thư viện
- Kết nối Internet trong lần cài đặt đầu tiên (để tải các mô hình NLP của Underthesea)

7.2 Các bước cài đặt

- **Bước 1: Tải hoặc clone dự án**

  Người dùng tải mã nguồn dự án và điều hướng vào thư mục chính:

  Cd Calendar_App
- **Bước 2: Tạo môi trường ảo (tuỳ chọn nhưng khuyến nghị)**

  Việc sử dụng môi trường ảo giúp tách biệt các thư viện của dự án:

  python -m venv venv

  venv\Scripts\activate
- **Bước 3: Cài đặt các thư viện cần thiết**

  Ứng dụng sử dụng nhiều thư viện NLP và Flask, có thể được cài đặt bằng lệnh:

  pip install -r requirements.txt
- **Bước 4 : Chuẩn bị dữ liệu huấn luyện**

  Ứng dụng sử dụng tệp dữ liệu:

  Dataset/Schedule_Data.csv

  Tệp này cần chứa ít nhất 2 cột:

  - **text**: Mô tả lịch trình bằng tiếng Việt
  - **label**: Loại lịch, gồm các nhóm như:

    meeting, study, appointment, shopping, entertainment, birthday, gym, yoga, piano

  Dữ liệu được sử dụng để huấn luyện mô hình phân loại lịch trình.
- Bước 5 : **Khởi chạy ứng dụng**

  Sau khi cài đặt đầy đủ, khởi động server Flask bằng lệnh:

python app.py
Ứng dụng sẽ chạy tại địa chỉ:
http://localhost:5000
Mở trình duyệt và truy cập địa chỉ trên để bắt đầu sử dụng hệ thống.

8. Kết luận và hướng phát triển

8.1 Kết luận

Đề tài đã xây dựng thành công **Ứng dụng Quản lý Lịch trình** tích hợp **xử lý ngôn ngữ tự nhiên tiếng Việt**, cho phép người dùng tạo lịch trình bằng câu văn tự nhiên và quản lý lịch một cách thuận tiện. Hệ thống gồm: mô hình phân loại lịch đạt độ chính xác **85–90%**, module NLP trích xuất thời gian – địa điểm – người tham gia, giao diện web trực quan và cơ sở dữ liệu SQLite. Ứng dụng hoạt động ổn định, phản hồi nhanh và đáp ứng đầy đủ các yêu cầu đề ra.

8.2 Hướng phát triển

Trong tương lai, hệ thống có thể mở rộng theo các hướng:

- **Nâng cấp NLP và mô hình ML** để tăng độ chính xác và hỗ trợ nhiều dạng câu phức tạp hơn.
- **Bổ sung chức năng nhắc lịch**, gửi thông báo qua email hoặc ứng dụng nhắn tin.
- **Phát triển phiên bản mobile**, đồng bộ dữ liệu giữa các thiết bị.
- **Cải thiện giao diện**, thêm chế độ xem dạng lịch (Calendar View).
- **Nâng cấp cơ sở dữ liệu** sang MySQL/PostgreSQL khi triển khai nhiều người dùng.
- **Triển khai lên cloud** để người dùng có thể truy cập từ mọi nơi.\

Tài Liệu tham khảo

[1] Flask Documentation – *Flask Web Framework*. Truy cập tại: https://flask.palletsprojects.com/

[2] Underthesea – *Vietnamese NLP Toolkit*. Truy cập tại: https://github.com/undertheseanlp/underthesea

[3] Scikit-learn – *Machine Learning in Python*. Truy cập tại: https://scikit-learn.org/

[4] Pandas Documentation – *Python Data Analysis Library*. Truy cập tại: https://pandas.pydata.org/

[5] SQLite Documentation – *Lightweight Database Engine*. Truy cập tại: https://www.sqlite.org

[6] Jurafsky & Martin (Speech and Language Processing). *Natural Language Processing Book*. Pearson Education.

Phụ lục

App.py

```
import json
import sqlite3
from datetime import datetime, timedelta
from pathlib import Path
from flask import Flask, render_template, request, jsonify
from vi_nlp import IntentClassifier, VietnameseNLP

# Initialize Flask app
app = Flask(__name__)
app.config["JSON_AS_ASCII"] = False

# Initialize NLP
classifier = IntentClassifier()
if not classifier.load():
    print("Training classifier...")
    classifier.train("./Dataset/Schedule_Data.csv")
```

```
nlp = VietnameseNLP(classifier)

# Database setup
DB_PATH = "schedules.db"



def init_db():
    """Initialize SQLite database"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute(
        """
        CREATE TABLE IF NOT EXISTS schedules (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            description TEXT,
            date TEXT,
            time TEXT,
            location TEXT,
            intent_category TEXT,
            participants TEXT,
            reminder_time TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            status TEXT DEFAULT 'active'
        )
    """
    )
```

```
    conn.commit()
    conn.close()



def get_schedules(status="active"):
    """Get all schedules"""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    c = conn.cursor()
    c.execute("SELECT * FROM schedules WHERE status = ? ORDER BY date,
time", (status,))
    schedules = [dict(row) for row in c.fetchall()]
    conn.close()
    return schedules



def add_schedule(
    title, description, date, time, location, intent, participants, reminder
):
    """Add new schedule"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute(
        """
        INSERT INTO schedules
        (title, description, date, time, location, intent_category, participants,
reminder_time)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """,
```

```
        (
            title,
            description,
            date,
            time,
            location,
            intent,
            json.dumps(participants),
            reminder,
        ),
    )
    conn.commit()
    schedule_id = c.lastrowid
    conn.close()
    return schedule_id


def update_schedule(schedule_id, **kwargs):
    """Update schedule"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()

    allowed_fields = [
        "title",
        "description",
        "date",
        "time",
        "location",
```

```
        "intent_category",
        "status",
    ]
    updates = {k: v for k, v in kwargs.items() if k in allowed_fields}

    if updates:
        set_clause = ", ".join([f"{k} = ?" for k in updates.keys()])
        values = list(updates.values()) + [schedule_id]
        c.execute(f"UPDATE schedules SET {set_clause} WHERE id = ?", values)
        conn.commit()

    conn.close()



def delete_schedule(schedule_id):
    """Delete schedule (soft delete)"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE schedules SET status = ? WHERE id = ?", ("deleted",
schedule_id))
    conn.commit()
    conn.close()



# Routes
@app.route("/")
def index():
    """Main page"""
```

```
    schedules = get_schedules("active")
    return render_template("index.html", schedules=schedules)



@app.route("/api/schedules", methods=["GET", "POST"])
def api_schedules():
    """API for schedules"""
    if request.method == "GET":
        schedules = get_schedules("active")
        return jsonify(schedules)

    elif request.method == "POST":
        data = request.get_json()
        schedule_id = add_schedule(
            title=data.get("title"),
            description=data.get("description"),
            date=data.get("date"),
            time=data.get("time"),
            location=data.get("location"),
            intent=data.get("intent"),
            participants=data.get("participants", []),
            reminder=data.get("reminder_time"),
        )
        return jsonify({"id": schedule_id, "message": "Schedule created"})



@app.route("/api/schedules/<int:schedule_id>", methods=["GET", "PUT",
"DELETE"])
```

```
def api_schedule(schedule_id):
    """API for single schedule"""
    if request.method == "GET":
        conn = sqlite3.connect(DB_PATH)
        conn.row_factory = sqlite3.Row
        c = conn.cursor()
        c.execute("SELECT * FROM schedules WHERE id = ?", (schedule_id,))
        schedule = c.fetchone()
        conn.close()
        if schedule:
            return jsonify(dict(schedule))
        return jsonify({"error": "Not found"}), 404

    elif request.method == "PUT":
        data = request.get_json()
        update_schedule(schedule_id, **data)
        return jsonify({"message": "Schedule updated"})

    elif request.method == "DELETE":
        delete_schedule(schedule_id)
        return jsonify({"message": "Schedule deleted"})



@app.route("/api/parse", methods=["POST"])
def api_parse():
    """Parse Vietnamese text into schedule"""
    try:
        data = request.get_json()
```

```
    text = data.get("text", "")

    if not text:
        return jsonify({"error": "No text provided"}), 400

    result = nlp.parse(text)

    # Convert for JSON response - ensure all values are JSON serializable
    intent_proba = {}
    if result.get("intent_proba"):
        intent_proba = {
            k: float(v) for k, v in result.get("intent_proba", {}).items()
        }

    response = {
        "original_text": result["original_text"],
        "intent": str(result["intent"]) if result["intent"] else None,
        "intent_proba": intent_proba,
        "time": result["time"],
        "location": result["location"],
        "date_context": result["date_context"],
        "participants": result["participants"],
        "notes": result["notes"],
    }

    return jsonify(response)
except Exception as e:
    import traceback
```

```
        traceback.print_exc()
        return jsonify({"error": f"Parse error: {str(e)}"}), 500


@app.route("/api/suggestions", methods=["POST"])
def api_suggestions():
    """Get schedule suggestions from parsed text"""
    try:
        data = request.get_json()
        text = data.get("text", "")

        if not text:
            return jsonify({"error": "No text provided"}), 400

        parsed = nlp.parse(text)

        # Build suggestions - ensure all values are JSON serializable
        suggestions = {
            "title": text[:100],  # First 100 chars as title
            "intent": str(parsed["intent"]) if parsed["intent"] else None,
            "location": parsed["location"][0] if parsed["location"] else "",
            "participants": parsed["participants"],
            "reminder_time": parsed["notes"],
            "time": int(parsed["time"][0]["hour"])
            if parsed["time"] and parsed["time"][0].get("hour")
            else None,
            "date_context": list(parsed["date_context"].keys()),
```

```
        }

        return jsonify(suggestions)
    except Exception as e:
        import traceback

        traceback.print_exc()
        return jsonify({"error": f"Suggestion error: {str(e)}"}), 500



# Error handlers
@app.errorhandler(404)
def not_found(error):
    return jsonify({"error": "Not found"}), 404



@app.errorhandler(500)
def server_error(error):
    return jsonify({"error": "Server error"}), 500



if __name__ == "__main__":
    init_db()
    print("Starting Calendar App Server...")
    app.run(debug=True, port=5000)
```

vi-nlp.py

```
from datetime import datetime, timedelta
import re
```

```
import json
import pickle
import os
from underthesea import word_tokenize, pos_tag, ner, classify
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
import pandas as pd
import numpy as np

class IntentClassifier:
    """Classify Vietnamese schedule text into intent categories"""

    def __init__(self, model_path="model_classifier.pkl"):
        self.model_path = model_path
        self.pipeline = None
        self.classes = None

    def load_data(self, csv_path="./Dataset/Schedule_Data.csv"):
        """Load training data from CSV"""
        try:
            data = pd.read_csv(csv_path, encoding='utf-8')
```

```
        if 'label' not in data.columns or 'text' not in data.columns:
            raise ValueError("CSV must have 'text' and 'label' columns")
        return data
    except Exception as e:
        print(f"Error loading data: {e}")
        return None

def vi_tokenize(self, text):
    """Tokenize Vietnamese text"""
    return word_tokenize(text, format="text").split()

def train(self, csv_path="./Dataset/Schedule_Data.csv"):
    """Train classification model"""
    data = self.load_data(csv_path)
    if data is None:
        return False

    X = data['text'].values
    y = data['label'].values

    # Use stratify only if each class has at least 2 samples
    stratify_arg = y if (y == y[0]).sum() > 1 and len(y) > 1 else None
    for label in set(y):
        if (y == label).sum() < 2:
            stratify_arg = None
            break

    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.2, random_state=42, stratify=stratify_arg
    )

    # Build pipeline
    self.pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(tokenizer=self.vi_tokenize, max_features=1000)),
        ('clf', MultinomialNB(alpha=1.0))
    ])

    self.pipeline.fit(X_train, y_train)
    self.classes = self.pipeline.named_steps['clf'].classes_

    # Evaluate
    y_pred = self.pipeline.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Model trained. Accuracy: {accuracy:.2%}")

    # Save model
    self.save()
    return True

def predict(self, text):
    """Predict intent category"""
    if self.pipeline is None:
        self.load()
    if self.pipeline is None:
        return None
    return self.pipeline.predict([text])[0]
```

```
def predict_proba(self, text):
    """Get probability for each category"""
    if self.pipeline is None:
        self.load()
    if self.pipeline is None:
        return {}
    proba = self.pipeline.predict_proba([text])[0]
    return {cat: float(prob) for cat, prob in zip(self.classes, proba)}

def save(self):
    """Save trained model"""
    if self.pipeline:
        with open(self.model_path, 'wb') as f:
            pickle.dump(self.pipeline, f)
        print(f"Model saved to {self.model_path}")

def load(self):
    """Load pre-trained model"""
    if os.path.exists(self.model_path):
        with open(self.model_path, 'rb') as f:
            self.pipeline = pickle.load(f)
        # Extract classes from the loaded classifier
        if self.pipeline and hasattr(self.pipeline.named_steps['clf'], 'classes_'):
            self.classes = self.pipeline.named_steps['clf'].classes_
        return True
    return False
```

```
class VietnameseNLP:
    """Vietnamese NLP for schedule parsing"""

    def __init__(self, classifier=None):
        self.classifier = classifier
        self.time_patterns = {
            'lúc': r'lúc\s+(\d{1,2})\s*h(?:\s*(\d{1,2}))?',
            'vào': r'vào\s+(\w+)',
            'từ_đến': r'từ\s+(\d{1,2})\s*h\s+đến\s+(\d{1,2})\s*h',
        }

    def extract_time(self, text):
        """Extract time from Vietnamese text"""
        times = []

        # Pattern: "lúc 10h sáng" or "lúc 14h"
        match = re.search(r'lúc\s+(\d{1,2})\s*h(?:\s*(sáng|chiều|tối|đêm))?', text,
re.IGNORECASE)
        if match:
            hour = int(match.group(1))
            period = match.group(2) if match.group(2) else None
            times.append({'hour': hour, 'period': period, 'raw': match.group(0)})

        # Pattern: "từ 14h đến 16h"
        match = re.search(r'từ\s+(\d{1,2})\s*h\s+đến\s+(\d{1,2})\s*h', text)
        if match:
            times.append({
                'start_hour': int(match.group(1)),
```

```
            'end_hour': int(match.group(2)),
            'raw': match.group(0)
        })

    return times

def extract_location(self, text):
    """Extract location from Vietnamese text"""
    locations = []

    # Pattern: "ở [location]"
    matches = re.findall(r'ở\s+([^,\.]+?)(?:\s*,|$)', text)
    locations.extend(matches)

    # Pattern: "tại [location]"
    matches = re.findall(r'tại\s+([^,\.]+?)(?:\s*,|$)', text)
    locations.extend(matches)

    return locations

def extract_date_context(self, text):
    """Extract date context (mai, hôm nay, etc.)"""
    date_keywords = {
        'hôm nay': 'today',
        'mai': 'tomorrow',
        'ngày mai': 'tomorrow',
        'hôm qua': 'yesterday',
        'sáng': 'morning',
```

```
            'chiều': 'afternoon',
            'tối': 'evening',
            'đêm': 'night',
        }

        found = {}
        for vi_text, en_text in date_keywords.items():
            if vi_text.lower() in text.lower():
                found[en_text] = True

        return found

    def extract_participants(self, text):
        """Extract people names from text"""
        # Use NER to find PERSON entities
        try:
            ner_results = ner(text)
            people = [token for token, label in ner_results if label == 'B-PER' or label ==
'I-PER']
            return people
        except:
            # Fallback: simple pattern matching
            pattern = r'(?:anh|chị|bác|ông|bà)\s+(\w+)'
            return re.findall(pattern, text)

    def extract_notes(self, text):
        """Extract reminder notes"""
        # Pattern: "nhắc trước [time]"
```

```
        match = re.search(r'nhắc\s+trước\s+(\d+)\s+(phút|giờ)', text)
        if match:
            return f"Remind {match.group(1)} {match.group(2)} before"
        return None

    def parse(self, text):
        """Parse Vietnamese schedule text into structured format"""
        result = {
            'original_text': text,
            'intent': None,
            'time': self.extract_time(text),
            'location': self.extract_location(text),
            'date_context': self.extract_date_context(text),
            'participants': self.extract_participants(text),
            'notes': self.extract_notes(text),
            'tokens': word_tokenize(text, format="text").split(),
        }

        # Classify intent
        if self.classifier:
            result['intent'] = self.classifier.predict(text)
            result['intent_proba'] = self.classifier.predict_proba(text)

        return result

# Training on startup
if __name__ == "__main__":
    clf = IntentClassifier()
```

```
    if clf.load():
        print("Loaded pre-trained model")
    else:
        print("Training new model...")
        clf.train("./Dataset/Schedule_Data.csv")

    # Test parsing
    nlp = VietnameseNLP(clf)
    test_texts = [
        "Nhắc tôi họp nhóm lúc 10h sáng mai ở phòng 304",
        "Hẹn tôi đi siêu thị vào chiều nay ở VinMart",
        "Nhắc tôi học Python từ 14h đến 16h ở nhà",
    ]

    for text in test_texts:
        result = nlp.parse(text)
        print(f"\n{text}")
        print(f"Intent: {result['intent']}")
        print(f"Time: {result['time']}")
        print(f"Location: {result['location']}")
```

templates/index.html

```
<!DOCTYPE html>
<html lang="vi">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Ứng dụng Quản lý Lịch trình</title>
    <style>
```

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  min-height: 100vh;
  padding: 20px;
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  background: white;
  border-radius: 10px;
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
  overflow: hidden;
}

header {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 30px;
  text-align: center;
}
```

```
header h1 {
  font-size: 2.5em;
  margin-bottom: 10px;
}

header p {
  font-size: 1.1em;
  opacity: 0.9;
}

.main-content {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
  padding: 30px;
}

.section {
  background: #f8f9fa;
  border-radius: 8px;
  padding: 20px;
  border-left: 4px solid #667eea;
}

.section h2 {
  color: #667eea;
  margin-bottom: 20px;
```

```
  font-size: 1.5em;
}

.form-group {
  margin-bottom: 15px;
}

label {
  display: block;
  margin-bottom: 5px;
  color: #333;
  font-weight: 500;
}

input,
textarea,
select {
  width: 100%;
  padding: 10px;
  border: 1px solid #ddd;
  border-radius: 5px;
  font-family: inherit;
  font-size: 1em;
  transition: border-color 0.3s;
}

input:focus,
textarea:focus,
```

```
select:focus {
  outline: none;
  border-color: #667eea;
  box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

textarea {
  resize: vertical;
  min-height: 80px;
}

button {
  width: 100%;
  padding: 12px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  border: none;
  border-radius: 5px;
  font-size: 1em;
  font-weight: 600;
  cursor: pointer;
  transition: transform 0.2s, box-shadow 0.2s;
}

button:hover {
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}
```

```
button:active {
  transform: translateY(0);
}

.parse-section {
  grid-column: 1 / -1;
}

.parse-result {
  background: white;
  border: 1px solid #ddd;
  border-radius: 5px;
  padding: 15px;
  margin-top: 15px;
  display: none;
}

.parse-result.show {
  display: block;
  animation: slideIn 0.3s ease;
}

@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateY(-10px);
  }
```

```
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.parse-result h3 {
  color: #667eea;
  margin-bottom: 10px;
  font-size: 1.1em;
}

.parse-result p {
  margin: 8px 0;
  color: #555;
  line-height: 1.6;
}

.schedules-list {
  grid-column: 1 / -1;
}

.schedule-item {
  background: white;
  border: 1px solid #ddd;
  border-radius: 5px;
  padding: 15px;
  margin-bottom: 15px;
```

```
  display: grid;
  grid-template-columns: 1fr auto;
  align-items: start;
  gap: 15px;
  transition: box-shadow 0.2s;
}

.schedule-item:hover {
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
}

.schedule-info {
  flex: 1;
}

.schedule-title {
  font-size: 1.2em;
  font-weight: 600;
  color: #333;
  margin-bottom: 8px;
}

.schedule-meta {
  font-size: 0.9em;
  color: #666;
  margin: 5px 0;
}
```

```
.schedule-badge {
  display: inline-block;
  background: #667eea;
  color: white;
  padding: 4px 8px;
  border-radius: 4px;
  font-size: 0.85em;
  margin-right: 5px;
}

.schedule-actions {
  display: flex;
  gap: 10px;
  flex-direction: column;
}

.btn-small {
  padding: 8px 12px;
  font-size: 0.9em;
  width: auto;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.2s;
}

.btn-edit {
  background: #28a745;
```

```
  color: white;
}

.btn-edit:hover {
  background: #218838;
}

.btn-delete {
  background: #dc3545;
  color: white;
}

.btn-delete:hover {
  background: #c82333;
}

.suggestion-items {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px;
  margin-top: 15px;
}

.suggestion-item {
  background: white;
  padding: 10px;
  border-radius: 5px;
  border: 1px solid #ddd;
```

```
}

.suggestion-label {
  font-weight: 600;
  color: #667eea;
  font-size: 0.9em;
}

.suggestion-value {
  color: #333;
  margin-top: 5px;
}

.loading {
  text-align: center;
  color: #667eea;
  font-style: italic;
}

.error {
  background: #f8d7da;
  color: #721c24;
  padding: 12px;
  border-radius: 5px;
  margin-bottom: 15px;
  display: none;
}
```

```
    .error.show {
      display: block;
    }

    @media (max-width: 768px) {
      .main-content {
        grid-template-columns: 1fr;
      }

      .schedule-item {
        grid-template-columns: 1fr;
      }

      .suggestion-items {
        grid-template-columns: 1fr;
      }

      header h1 {
        font-size: 1.8em;
      }
    }
  </style>
</head>
<body>
  <div class="container">
    <header>
      <h1>📅 Quản lý Lịch trình</h1>
      <p>Ứng dụng quản lý lịch cá nhân với xử lý Tiếng Việt thông minh</p>
```

```
</header>

<div class="main-content">
  <!-- Parse Section -->
  <div class="parse-section section">
    <h2>🤖 Phân tích Văn bản</h2>
    <div class="error" id="parseError"></div>

    <div class="form-group">
      <label for="parseText">Nhập lịch trình bằng Tiếng Việt:</label>
      <textarea
        id="parseText"
        placeholder="VD: Nhắc tôi họp nhóm lúc 10h sáng mai ở phòng 304"
      ></textarea>
    </div>

    <button onclick="parseText()">Phân tích</button>

    <div class="parse-result" id="parseResult">
      <h3>Kết quả phân tích:</h3>
      <div id="parseResultContent"></div>
      <button onclick="useAsSchedule()" style="margin-top: 15px">
        Tạo lịch từ kết quả này
      </button>
    </div>
  </div>

  <!-- Add Schedule Section -->
```

```
<div class="section">
  <h2>➕ Thêm lịch trình</h2>
  <div class="error" id="addError"></div>

  <div class="form-group">
    <label for="title">Tiêu đề:</label>
    <input type="text" id="title" placeholder="VD: Họp nhóm" />
  </div>

  <div class="form-group">
    <label for="date">Ngày:</label>
    <input type="date" id="date" />
  </div>

  <div class="form-group">
    <label for="time">Giờ:</label>
    <input type="time" id="time" />
  </div>

  <div class="form-group">
    <label for="location">Địa điểm:</label>
    <input type="text" id="location" placeholder="VD: Phòng 304" />
  </div>

  <div class="form-group">
    <label for="category">Loại lịch:</label>
    <select id="category">
      <option value="">-- Chọn --</option>
```

```
      <option value="meeting">Họp</option>
      <option value="study">Học tập</option>
      <option value="appointment">Cuộc hẹn</option>
      <option value="shopping">Mua sắm</option>
      <option value="entertainment">Giải trí</option>
      <option value="birthday">Sinh nhật</option>
      <option value="gym">Tập luyện</option>
    </select>
  </div>

  <div class="form-group">
    <label for="reminder">Nhắc nhở trước:</label>
    <input type="text" id="reminder" placeholder="VD: 15 phút" />
  </div>

  <button onclick="addSchedule()">Thêm lịch</button>
</div>

<!-- Schedules List -->
<div class="schedules-list section">
  <h2>📋 Danh sách lịch trình</h2>
  <div style="display: flex; gap: 8px; margin-bottom: 12px">
    <input
      id="searchInput"
      type="text"
      placeholder="Tìm kiếm theo tiêu đề, địa điểm, loại..."
      style="
        flex: 1;
```

```
            padding: 8px;
            border-radius: 6px;
            border: 1px solid #ddd;
          "
        />
        <button onclick="loadSchedules()" style="width: 140px">
          Làm mới
        </button>
      </div>
      <div id="schedulesList"></div>
    </div>
  </div>
</div>
<!-- Reminder popup modal -->
<div
  id="reminderModal"
  style="
    display: none;
    position: fixed;
    right: 20px;
    bottom: 20px;
    z-index: 2000;
    max-width: 320px;
    background: #fff;
    border-radius: 8px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
    overflow: hidden;
  "
```

```
>
  <div style="padding: 12px; border-left: 6px solid #667eea">
    <div
      id="reminderTitle"
      style="font-weight: 700; color: #333; margin-bottom: 6px"
    ></div>
    <div
      id="reminderBody"
      style="color: #555; font-size: 0.95em; margin-bottom: 10px"
    ></div>
    <div style="display: flex; gap: 8px">
      <button
        id="reminderDismiss"
        style="
          flex: 1;
          padding: 8px;
          background: #dc3545;
          color: #fff;
          border: none;
          border-radius: 6px;
          cursor: pointer;
        "
      >
        Đã nhận
      </button>
      <button
        id="reminderSnooze"
        style="
```

```
          flex: 1;
          padding: 8px;
          background: #ffc107;
          color: #111;
          border: none;
          border-radius: 6px;
          cursor: pointer;
        "
      >
        Hoãn 5 phút
      </button>
    </div>
  </div>
</div>
<!-- Edit modal -->
<div
  id="editModal"
  style="
    display: none;
    position: fixed;
    left: 50%;
    top: 50%;
    transform: translate(-50%, -50%);
    z-index: 3000;
    width: 400px;
    background: #fff;
    border-radius: 8px;
    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
```

```
  "
>
  <div style="padding: 16px">
    <h3 style="margin-bottom: 8px; color: #333">Sửa lịch</h3>
    <input type="hidden" id="edit_id" />
    <div style="margin-bottom: 8px">
      <input
        id="edit_title"
        placeholder="Tiêu đề"
        style="
          width: 100%;
          padding: 8px;
          border: 1px solid #ddd;
          border-radius: 6px;
        "
      />
    </div>
    <div style="display: flex; gap: 8px; margin-bottom: 8px">
      <input
        id="edit_date"
        type="date"
        style="
          flex: 1;
          padding: 8px;
          border: 1px solid #ddd;
          border-radius: 6px;
        "
      /><input
```

```
    id="edit_time"
    type="time"
    style="
      width: 120px;
      padding: 8px;
      border: 1px solid #ddd;
      border-radius: 6px;
    "
  />
</div>
<div style="margin-bottom: 8px">
  <input
    id="edit_location"
    placeholder="Địa điểm"
    style="
      width: 100%;
      padding: 8px;
      border: 1px solid #ddd;
      border-radius: 6px;
    "
  />
</div>
<div style="margin-bottom: 8px">
  <select
    id="edit_category"
    style="
      width: 100%;
      padding: 8px;
```

```
      border: 1px solid #ddd;
      border-radius: 6px;
    "
  >
    <option value="">-- Chọn --</option>
    <option value="meeting">Họp</option>
    <option value="study">Học</option>
    <option value="appointment">Cuộc hẹn</option>
    <option value="shopping">Mua sắm</option>
    <option value="entertainment">Giải trí</option>
  </select>
</div>
<div style="margin-bottom: 8px">
  <input
    id="edit_reminder"
    placeholder="Nhắc trước (VD: 15 phút)"
    style="
      width: 100%;
      padding: 8px;
      border: 1px solid #ddd;
      border-radius: 6px;
    "
  />
</div>
<div style="margin-bottom: 8px">
  <input
    id="edit_participants"
    placeholder="Người tham gia (phân tách bằng ,)"
```

```
      style="
        width: 100%;
        padding: 8px;
        border: 1px solid #ddd;
        border-radius: 6px;
      "
    />
  </div>
  <div style="display: flex; gap: 8px">
    <button
      onclick="saveEdit()"
      style="
        flex: 1;
        background: #28a745;
        color: #fff;
        padding: 10px;
        border-radius: 6px;
        border: none;
      "
    >
      Lưu
    </button>
    <button
      onclick="closeEditModal()"
      style="
        flex: 1;
        background: #6c757d;
        color: #fff;
```

```
            padding: 10px;
            border-radius: 6px;
            border: none;
          "
        >
          Hủy
        </button>
      </div>
    </div>
  </div>

  <script>
    // Schedules cache for search/edit
    let schedulesCache = [];
    // Reminder helper: track shown reminders in localStorage to avoid repeated
popups
    const REMINDERS_KEY = "shownReminders_v1";
    function getShownReminders() {
      try {
        return JSON.parse(localStorage.getItem(REMINDERS_KEY) || "[]");
      } catch (e) {
        return [];
      }
    }
    function addShownReminder(key) {
      const arr = getShownReminders();
      arr.push(key);
      localStorage.setItem(REMINDERS_KEY, JSON.stringify(arr));
```

```
}

// Parse reminder text like "15 phút" or "Remind 15 phút before" => minutes
function parseReminderToMinutes(rem) {
  if (!rem) return 0;
  rem = rem.toString().toLowerCase();
  const m = rem.match(/(\d+)\s*phút/);
  if (m) return parseInt(m[1]);
  const h = rem.match(/(\d+)\s*giờ/);
  if (h) return parseInt(h[1]) * 60;
  // fallback: try number only
  const n = rem.match(/(\d+)/);
  if (n) return parseInt(n[1]);
  return 0;
}

// Build Date object from schedule's date and time
function scheduleDateTime(schedule) {
  if (!schedule.date) return null;
  let timeStr = schedule.time || "";
  // Normalize formats like '10:00' or '10' or '10:00:00'
  if (/^[0-9]{1,2}$/.test(timeStr))
    timeStr = timeStr.padStart(2, "0") + ":00";
  if (timeStr && !timeStr.includes(":")) timeStr = timeStr + ":00";
  const dtStr = schedule.date + (timeStr ? "T" + timeStr : "T00:00:00");
  const d = new Date(dtStr);
  if (isNaN(d)) return null;
  return d;
```

```
}

// Show browser notification + in-page popup
async function showReminderPopup(schedule, reminderAt) {
  const key = `${schedule.id}__${reminderAt.toISOString()}`;
  if (getShownReminders().includes(key)) return; // already shown

  // Browser Notification
  if (window.Notification && Notification.permission === "granted") {
    try {
      new Notification(schedule.title || "Nhắc lịch", {
        body: `${schedule.location || ""} — ${schedule.time || ""}`,
      });
    } catch (e) {
      console.warn("Notification failed", e);
    }
  }

  // Play beep via WebAudio
  try {
    const ctx = new (window.AudioContext || window.webkitAudioContext)();
    const o = ctx.createOscillator();
    const g = ctx.createGain();
    o.type = "sine";
    o.frequency.value = 880;
    g.gain.value = 0.05;
    o.connect(g);
    g.connect(ctx.destination);
```

```
  o.start();
  setTimeout(() => {
    o.stop();
    ctx.close();
  }, 600);
} catch (e) {
  /* ignore */
}

// In-page modal
const modal = document.getElementById("reminderModal");
document.getElementById("reminderTitle").textContent =
  schedule.title || "Nhắc lịch";
document.getElementById("reminderBody").textContent = `${
  schedule.date || ""
} ${schedule.time || ""} — ${schedule.location || ""}`;
modal.style.display = "block";

function dismiss() {
  modal.style.display = "none";
  addShownReminder(key);
  removeHandlers();
}
function snooze() {
  modal.style.display = "none";
  addShownReminder(key);
  removeHandlers();
  // schedule a local re-notify after 5 minutes
```

```
    setTimeout(() => {
      showReminderPopup(schedule, new Date(Date.now() + 5 * 60 * 1000));
    }, 5 * 60 * 1000);
  }

  function removeHandlers() {
    document
      .getElementById("reminderDismiss")
      .removeEventListener("click", dismiss);
    document
      .getElementById("reminderSnooze")
      .removeEventListener("click", snooze);
  }

  document
    .getElementById("reminderDismiss")
    .addEventListener("click", dismiss);
  document
    .getElementById("reminderSnooze")
    .addEventListener("click", snooze);
}

// Poll schedules and check for upcoming reminders
let reminderPollInterval = null;
async function pollReminders() {
  try {
    const res = await fetch("/api/schedules");
    if (!res.ok) return;
```

```
    const schedules = await res.json();
    const now = new Date();
    for (const s of schedules) {
      const eventDt = scheduleDateTime(s);
      if (!eventDt) continue;
      // compute reminder minutes
      const remMinutes = parseReminderToMinutes(
        s.reminder_time || s.notes || ""
      );
      const reminderAt = new Date(
        eventDt.getTime() - remMinutes * 60 * 1000
      );
      // If reminderAt is within next 60 seconds and not in future far, show
      if (
        reminderAt <= new Date(now.getTime() + 60 * 1000) &&
        reminderAt >= new Date(now.getTime() - 60 * 1000)
      ) {
        await showReminderPopup(s, reminderAt);
      }
    }
  } catch (e) {
    console.warn("pollReminders error", e);
  }
}

function startReminderPolling() {
  if (reminderPollInterval) clearInterval(reminderPollInterval);
  // initial poll
```

```
  pollReminders();
  reminderPollInterval = setInterval(pollReminders, 30 * 1000); // every 30s
}

// Request permission and start polling on load
window.addEventListener("load", () => {
  if (window.Notification && Notification.permission !== "granted") {
    try {
      Notification.requestPermission();
    } catch (e) {}
  }
  startReminderPolling();
});
let currentParsedData = null;

async function parseText() {
  const text = document.getElementById("parseText").value.trim();
  const errorDiv = document.getElementById("parseError");

  if (!text) {
    errorDiv.textContent = "Vui lòng nhập văn bản";
    errorDiv.classList.add("show");
    return;
  }

  errorDiv.classList.remove("show");

  try {
```

```
    const response = await fetch("/api/parse", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text }),
    });

    if (!response.ok) throw new Error("Parse failed");

    const data = await response.json();
    currentParsedData = data;

    displayParseResult(data);
  } catch (error) {
    errorDiv.textContent = "Lỗi: " + error.message;
    errorDiv.classList.add("show");
  }
}

function displayParseResult(data) {
  const resultDiv = document.getElementById("parseResult");
  const contentDiv = document.getElementById("parseResultContent");

  let html = `
        <p><strong>Ý định:</strong> ${
          data.intent || "Không xác định"
        }</p>
        <p><strong>Thời gian:</strong> ${
          data.time.length > 0
```

```
            ? JSON.stringify(data.time)
            : "Không tìm thấy"
          }</p>
          <p><strong>Địa điểm:</strong> ${
            data.location.length > 0
            ? data.location.join(", ")
            : "Không tìm thấy"
          }</p>
          <p><strong>Người tham gia:</strong> ${
            data.participants.length > 0
            ? data.participants.join(", ")
            : "Không tìm thấy"
          }</p>
          <p><strong>Ghi chú:</strong> ${data.notes || "Không có"}</p>
        `;

  contentDiv.innerHTML = html;
  resultDiv.classList.add("show");
}

async function useAsSchedule() {
  if (!currentParsedData) return;

  const data = currentParsedData;
  document.getElementById("title").value = document
    .getElementById("parseText")
    .value.substring(0, 100);
  document.getElementById("category").value = data.intent || "";
```

```
  if (data.location.length > 0) {
    document.getElementById("location").value = data.location[0];
  }
  if (data.time.length > 0 && data.time[0].hour) {
    document.getElementById("time").value =
      String(data.time[0].hour).padStart(2, "0") + ":00";
  }
}

async function addSchedule() {
  const title = document.getElementById("title").value.trim();
  const date = document.getElementById("date").value;
  const time = document.getElementById("time").value;
  const location = document.getElementById("location").value.trim();
  const intent = document.getElementById("category").value;
  const reminder = document.getElementById("reminder").value.trim();

  const errorDiv = document.getElementById("addError");

  if (!title || !date) {
    errorDiv.textContent = "Vui lòng điền tiêu đề và ngày";
    errorDiv.classList.add("show");
    return;
  }

  errorDiv.classList.remove("show");

  try {
```

```
    const response = await fetch("/api/schedules", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        title,
        date,
        time,
        location,
        intent,
        reminder_time: reminder,
        participants: [],
      }),
    });

    if (!response.ok) throw new Error("Failed to add schedule");

    document.getElementById("title").value = "";
    document.getElementById("date").value = "";
    document.getElementById("time").value = "";
    document.getElementById("location").value = "";
    document.getElementById("category").value = "";
    document.getElementById("reminder").value = "";

    loadSchedules();
  } catch (error) {
    errorDiv.textContent = "Lỗi: " + error.message;
    errorDiv.classList.add("show");
  }
```

```
}

async function loadSchedules() {
  try {
    const response = await fetch("/api/schedules");
    const schedules = await response.json();
    schedulesCache = schedules;
    renderSchedules(schedules);
  } catch (error) {
    console.error("Error loading schedules:", error);
  }
}

function renderSchedules(list) {
  const listDiv = document.getElementById("schedulesList");
  if (!list || list.length === 0) {
    listDiv.innerHTML =
      '<p style="text-align: center; color: #999;">Chưa có lịch trình nào</p>';
    return;
  }

  listDiv.innerHTML = list
    .map(
      (s) => `
          <div class="schedule-item">
              <div class="schedule-info">
                  <div class="schedule-title">${s.title}</div>
                  ${
```

```
            s.date
              ? `<div class="schedule-meta">${s.date} ${
                  s.time || ""
                }</div>`
              : ""
          }
          ${
            s.location
              ? `<div class="schedule-meta">${s.location}</div>`
              : ""
          }
          ${
            s.intent_category
              ? `<span class="schedule-badge">${s.intent_category}</span>`
              : ""
          }
          ${
            s.reminder_time
              ? `<div class="schedule-meta">${s.reminder_time}</div>`
              : ""
          }
        </div>
        <div class="schedule-actions">
          <button class="btn-small btn-edit" onclick="openEditModal(${
            s.id
          })">Sửa</button>
          <button class="btn-small btn-delete" onclick="deleteSchedule(${
            s.id
```

```
                })">Xóa</button>
            </div>
        </div>
      `
    )
    .join("");
}

// Search handler
document.getElementById("searchInput").addEventListener("input", (e) => {
  const q = e.target.value.trim().toLowerCase();
  if (!q) return renderSchedules(schedulesCache);
  const filtered = schedulesCache.filter((s) => {
    return (
      (s.title || "").toLowerCase().includes(q) ||
      (s.location || "").toLowerCase().includes(q) ||
      (s.intent_category || "").toLowerCase().includes(q) ||
      (s.date || "").toLowerCase().includes(q)
    );
  });
  renderSchedules(filtered);
});

async function deleteSchedule(id) {
  if (!confirm("Bạn chắc chắn muốn xóa?")) return;

  try {
    const response = await fetch(`/api/schedules/${id}`, {
```

```
      method: "DELETE",
    });
    if (response.ok) {
      loadSchedules();
    }
  } catch (error) {
    alert("Lỗi: " + error.message);
  }
}

// Edit modal handlers
function openEditModal(id) {
  const s = schedulesCache.find((x) => x.id === id);
  if (!s) return alert("Không tìm thấy lịch");
  document.getElementById("edit_id").value = s.id;
  document.getElementById("edit_title").value = s.title || "";
  document.getElementById("edit_date").value = s.date || "";
  document.getElementById("edit_time").value = s.time || "";
  document.getElementById("edit_location").value = s.location || "";
  document.getElementById("edit_category").value =
    s.intent_category || "";
  document.getElementById("edit_reminder").value = s.reminder_time || "";
  try {
    document.getElementById("edit_participants").value = JSON.parse(
      s.participants || "[]"
    ).join(", ");
  } catch (e) {
    document.getElementById("edit_participants").value =
```

```
      s.participants || "";
  }
  document.getElementById("editModal").style.display = "block";
}

function closeEditModal() {
  document.getElementById("editModal").style.display = "none";
}

async function saveEdit() {
  const id = document.getElementById("edit_id").value;
  const title = document.getElementById("edit_title").value.trim();
  const date = document.getElementById("edit_date").value;
  const time = document.getElementById("edit_time").value;
  const location = document.getElementById("edit_location").value.trim();
  const intent = document.getElementById("edit_category").value;
  const reminder = document.getElementById("edit_reminder").value.trim();
  const participantsRaw = document
    .getElementById("edit_participants")
    .value.trim();
  const participants = participantsRaw
    ? participantsRaw
        .split(",")
        .map((s) => s.trim())
        .filter(Boolean)
    : [];

  try {
```

```
    const payload = {
      title,
      date,
      time,
      location,
      intent_category: intent,
      reminder_time: reminder,
      participants: JSON.stringify(participants),
    };
    const res = await fetch(`/api/schedules/${id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),
    });
    if (!res.ok) throw new Error("Update failed");
    closeEditModal();
    loadSchedules();
  } catch (e) {
    alert("Lỗi cập nhật: " + e.message);
  }
}

// Initialize
window.addEventListener("load", () => {
  loadSchedules();
  // Set today's date as default
  const today = new Date().toISOString().split("T")[0];
  document.getElementById("date").value = today;
```

```
    });
  </script>
 </body>
</html>
```

Link dự án : https://github.com/QUOCDIEN1234/QllichcanhaSGU.git

Test Case

Test case	Mong muốn	Đạt được
"Nhắc tôi học lập trình web lúc 14h chiều mai ở nhà, nhắc trước 20 phút",study	Ý định: study Thời gian: [{"hour":14,"period":"chiều","raw":"lúc 14h chiều"}] Địa điểm: nhà Người tham gia: Không tìm thấy Ghi chú: Remind 20 phút before	Ý định: study Thời gian: [{"hour":14,"period":"chiều","raw":"lúc 14h chiều"}] Địa điểm: nhà Người tham gia: Không tìm thấy Ghi chú: Remind 20 phút before
"Hẹn tôi học đàn piano lúc 17h chiều nay ở trung tâm âm nhạc, nhắc trước 15 phút",piano	Ý định: study Thời gian: [{"hour":17,"period":"chiều","raw":"lúc 17h chiều"}] Địa điểm: trung tâm âm nhạc Người tham gia: Không tìm thấy Ghi chú: Remind 15 phút before	Ý định: study Thời gian: [{"hour":17,"period":"chiều","raw":"lúc 17h chiều"}] Địa điểm: trung tâm âm nhạc Người tham gia: Không tìm thấy Ghi chú: Remind 15 phút before
"Nhắc tôi làm bài tập toán lúc 15h chiều nay ở nhà, nhắc trước 15 phút",study	Ý định: study Thời gian: [{"hour":15,"period":"chiều","raw":"lúc 15h chiều"}] Địa điểm: nhà Người tham gia: Không tìm thấy Ghi chú: Remind 15 phút before	Ý định: study Thời gian: [{"hour":15,"period":"chiều","raw":"lúc 15h chiều"}] Địa điểm: nhà Người tham gia: Không tìm thấy Ghi chú: Remind 15 phút before

"Nhắc tôi họp nhóm thiết kế lúc 11h sáng mai ở phòng 306, nhắc trước 15 phút", meeting	Ý định: meeting Thời gian: [{"hour":11,"period":"sáng", "raw":"lúc 11h sáng"}] Địa điểm: phòng 306 Người tham gia: Không tìm thấy Ghi chú: Remind 15 phút before	Ý định: meeting Thời gian: [{"hour":11,"period":"sáng" ,"raw":"lúc 11h sáng"}] Địa điểm: phòng 306 Người tham gia: Không tìm thấy Ghi chú: Remind 15 phút before
"Hẹn tôi đọc sách tiếng Anh lúc 20h tối nay ở thư viện, nhắc trước 10 phút",study	Ý định: study Thời gian: [{"hour":20,"period":"tối","raw":"lúc 20h tối"}] Địa điểm: thư viện Người tham gia: Không tìm thấy Ghi chú: Remind 10 phút before	Ý định: study Thời gian: [{"hour":20,"period":"tối"," raw":"lúc 20h tối"}] Địa điểm: thư viện Người tham gia: Không tìm thấy Ghi chú: Remind 10 phút before
Nhắc tôi họp nhóm lúc 10h sáng mai tại phòng C304	Ý định: meeting Thời gian: [{"hour":10,"minute":0,"period":"sáng","raw":"lúc 10h sáng"}] Địa điểm: phòng C304 Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":10,"minute":0,"period":"sáng","raw":"lúc 10h sáng"}] Địa điểm: phòng C304 Ghi chú: Không có
Nhắc tôi gặp gỡ khách hàng vào 14 giờ chiều thứ 2 tại	Ý định: meeting Thời gian: [{"hour":14,"minute":0,"period":"sáng","raw":"lúc 14 giờ chiều"}] Địa điểm: quán cà phê tầng 5 Ghi chú: Không có	Ý định: study Thời gian: Không tìm thấy Địa điểm: quán cà phê tầng 5 Ghi chú: Không có

quán cà phê tầng 5		
nhắc tôi trả lời email lúc 9h30 sáng nay tại bàn làm việc	Ý định: meeting Thời gian: [{"hour":9,"minute":30,"period":"sáng","raw":"9h30 sáng"}] Địa điểm: bàn làm việc Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":9,"minute":30,"period":"sáng","raw":"9h30 sáng"}] Địa điểm: bàn làm việc Ghi chú: Không có
Nhắc tôi tham gia buổi đào tạo vào 16 giờ 30 phút ngày kia tại hội trường lớn	Ý định: study Thời gian: [{"hour":16,"minute":30,"period":"chiều","raw":"16h30 chiều"}] Địa điểm: hội trường lớn Ghi chú: Không có	Ý định: study Thời gian: Không tìm thấy Địa điểm: hội trường lớn Ghi chú: Không có
Nhắc tôi gọi điện cho anh Nam lúc 11 giờ trưa tại sân thượng	Ý định: meeting Thời gian: [{"hour":11,"minute":0,"period":null,"raw":"lúc 11 giờ"}] Địa điểm: sân thượng Ghi chú: Không có	Ý định: study Thời gian: [{"hour":11,"minute":0,"period":null,"raw":"lúc 11 giờ"}] Địa điểm: sân thượng Ghi chú: Không có
Nhắc tôi hoàn thành báo cáo trước 17h00 hôm nay tại	Ý định: meeting Thời gian: [{"hour":17,"minute":0,"period":null,"raw":"17h00"}] Địa điểm: nhà riêng Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":17,"minute":0,"period":null,"raw":"17h00"}] Địa điểm: nhà riêng Ghi chú: Không có

nhà riêng		
Nhắc tôi luyện tập Yoga vào 6h sáng mỗi Thứ 4 tại Phòng tập KingSport	Ý định: excircise Thời gian: [{"hour":6,"minute":0,"period":sáng,"raw":"vào 6 giờ sáng"}] Địa điểm: Phòng tập KingSport Ghi chú: Không có	Ý định: meeting Thời gian: Không tìm thấy Địa điểm: Phòng tập KingSport Ghi chú: Không có
Nhắc tôi mua quà sinh nhật lúc 18 giờ tối nay tại Trung tâm thương mại	Ý định: shopping Thời gian: [{"hour":18,"minute":0,"period":"tối","raw":"lúc 18 giờ tối"}] Địa điểm: Trung tâm thương mại Ghi chú: Không có	Ý định: study Thời gian: [{"hour":18,"minute":0,"period":"tối","raw":"lúc 18 giờ tối"}] Địa điểm: Trung tâm thương mại Ghi chú: Không có
Nhắc tôi đưa xe đi bảo dưỡng lúc 7 giờ mai tại Garage Phúc Hưng	Ý định: meeting Thời gian: [{"hour":7,"minute":0,"period":null,"raw":"lúc 7 giờ"}] Địa điểm: Garage Phúc Hưng Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":7,"minute":0,"period":null,"raw":"lúc 7 giờ"}] Địa điểm: Garage Phúc Hưng Ghi chú: Không có
Nhắc tôi đọc sách chuyên ngành vào 21 giờ 45 phút tại	Ý định: study Thời gian: [{"hour":21,"minute":45,"period":null,"raw":"vào 21 giờ 45 phút"}] Địa điểm: phòng khách Ghi chú: Không có	Ý định: study Thời gian: Không tìm thấy Địa điểm: phòng khách Ghi chú: Không có

phòng khách		
Nhắc tôi thanh toán hóa đơn điện lúc 12 giờ trưa mai tại ngân hàng Vietcombank	Ý định: shopping Thời gian: [{"hour":12,"minute":0,"period":null,"raw":"lúc 12 giờ"}] Địa điểm: ngân hàng Vietcombank Ghi chú: Không có	Ý định: study Thời gian: [{"hour":12,"minute":0,"period":null,"raw":"lúc 12 giờ"}] Địa điểm: ngân hàng Vietcombank Ghi chú: Không có
Nhắc tôi ăn trưa lúc 12h00 tại căng tin công ty	Ý định: meeting Thời gian: [{"hour":12,"minute":0,"period":null,"raw":"12h00"}] Địa điểm: căng tin công ty Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":12,"minute":0,"period":null,"raw":"12h00"}] Địa điểm: căng tin công ty Ghi chú: Không có
Nhắc tôi nhặt quần áo khô lúc 15h15 tại sân phơi	Ý định: meeting Thời gian: [{"hour":15,"minute":15,"period":null,"raw":"15h15"}] Địa điểm: sân phơi Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":15,"minute":15,"period":null,"raw":"15h15"}] Địa điểm: sân phơi Ghi chú: Không có
Nhắc tôi tham dự lễ khai trương lúc 19h00 tối Thứ Bảy tại Nhà hàng Trống Đồng	Ý định: study Thời gian: [{"hour":19,"minute":0,"period":"tối","raw":"19h00 tối"}] Địa điểm: Nhà hàng Trống Đồng Ghi chú: Không có	Ý định: study Thời gian: [{"hour":19,"minute":0,"period":"tối","raw":"19h00 tối"}] Địa điểm: Nhà hàng Trống Đồng Ghi chú: Không có
Nhắc tôi viết tóm tắt	Ý định: study	Ý định: study

cuộc họp lúc 14 giờ tại thư viện thành phố	Thời gian: [{"hour":14,"minute":0,"period":null,"raw":"lúc 14 giờ"}] Địa điểm: thư viện thành phố Ghi chú: Không có	Thời gian: [{"hour":14,"minute":0,"period":null,"raw":"lúc 14 giờ"}] Địa điểm: thư viện thành phố Ghi chú: Không có
Nhắc tôi chạy deadline trước 23h59 đêm nay tại máy tính cá nhân	Ý định: study Thời gian: [{"hour":23,"minute":59,"period":"đêm","raw":"23h59 đêm"}] Địa điểm: máy tính cá nhân Ghi chú: Không có	Ý định: study Thời gian: [{"hour":23,"minute":59,"period":"đêm","raw":"23h59 đêm"}] Địa điểm: máy tính cá nhân Ghi chú: Không có
Nhắc tôi mua sắm cây cảnh lúc 10 giờ sáng Chủ Nhật tại chợ hoa Hồ Thị Kỷ	Ý định: shopping Thời gian: [{"hour":10,"minute":0,"period":"sáng","raw":"lúc 10 giờ sáng"}] Địa điểm: chợ hoa Hồ Thị Kỷ Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":10,"minute":0,"period":"sáng","raw":"lúc 10 giờ sáng"}] Địa điểm: chợ hoa Hồ Thị Kỷ Ghi chú: Không có
Nhắc tôi kiểm tra kho hàng lúc 11 giờ 30 phút trưa tại nhà kho phía sau	Ý định: meeting Thời gian: [{"hour":11,"minute":0,"period":null,"raw":"lúc 11 giờ"}] Địa điểm: nhà kho phía sau Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":11,"minute":0,"period":null,"raw":"lúc 11 giờ"}] Địa điểm: nhà kho phía sau Ghi chú: Không có
Nhắc tôi tham	Ý định: study	Ý định: study

gia lớp học nhảy lúc 20h tối Thứ Sáu tại Studio của cô Mai	Thời gian: [{"hour":20,"minute":0,"period":"tối","raw":"lúc 20h tối"}] Địa điểm: Studio của cô Mai Ghi chú: Không có	Thời gian: [{"hour":20,"minute":0,"period":"tối","raw":"lúc 20h tối"}] Địa điểm: Studio của cô Mai Ghi chú: Không có
Nhắc tôi chuẩn bị bài thuyết trình lúc 14 giờ Thứ Tư tại máy tính xách tay	Ý định: meeting Thời gian: [{"hour":14,"minute":0,"period":null,"raw":"lúc 14 giờ"}] Địa điểm: máy tính xách tay Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":14,"minute":0,"period":null,"raw":"lúc 14 giờ"}] Địa điểm: máy tính xách tay Ghi chú: Không có
Nhắc tôi gửi tài liệu trước 10h30 qua Google Drive	Ý định: meeting Thời gian: [{"hour":10,"minute":30,"period":null,"raw":"10h30"}] Địa điểm: Không tìm thấy Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":10,"minute":30,"period":null,"raw":"10h30"}] Địa điểm: Không tìm thấy Ghi chú: Không có
Nhắc tôi họp trực tuyến lúc 16h30 qua phần mềm Zoom	Ý định: meeting Thời gian: [{"hour":16,"minute":30,"period":null,"raw":"16h30"}] Địa điểm: Không tìm thấy Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":16,"minute":30,"period":null,"raw":"16h30"}] Địa điểm: Không tìm thấy Ghi chú: Không có
Hẹn tôi đi mua sắm thực phẩm lúc 17h chiều	Ý định: shopping Thời gian: [{"hour":17,"minute":0,"period":"chiều","raw":"lúc 17h chiều"}] Địa điểm: siêu thị BigC Ghi chú: Không có	Ý định: shopping Thời gian: [{"hour":17,"minute":0,"period":"chiều","raw":"lúc 17h chiều"}] Địa điểm: siêu thị BigC Ghi chú: Không có

nay ở siêu thị BigC		
Nhắc tôi đi khám răng định kỳ lúc 9h sáng mai ở nha khoa Smile, nhắc trước 20 phút	Ý định: appointment Thời gian: [{"hour":9,"minute":0,"period":"sáng","raw":"lúc 9h sáng"}] Địa điểm: nha khoa Smile Ghi chú: Remind 20 phút before	Ý định: appointment Thời gian: [{"hour":9,"minute":0,"period":"sáng","raw":"lúc 9h sáng"}] Địa điểm: nha khoa Smile Ghi chú: Remind 20 phút before
Hẹn tôi đi mua sắm quần áo lúc 16h chiều nay ở Vincom	Ý định: shopping Thời gian: [{"hour":16,"minute":0,"period":"chiều","raw":"lúc 16h chiều"}] Địa điểm: Vincom Ghi chú: Không có	Ý định: shopping Thời gian: [{"hour":16,"minute":0,"period":"chiều","raw":"lúc 16h chiều"}] Địa điểm: Vincom Ghi chú: Không có
Hẹn tôi đi xem phim tối nay lúc 19h ở rạp CGV	Ý định: entertainment Thời gian: [{"hour":19,"minute":0,"period":null,"raw":"lúc 19h"}] Địa điểm: rạp CGV Ghi chú: Không có	Ý định: entertainment Thời gian: [{"hour":19,"minute":0,"period":null,"raw":"lúc 19h"}] Địa điểm: rạp CGV Ghi chú: Không có
Nhắc tôi chuẩn bị báo cáo tài chính lúc 15h chiều nay ở văn phòng	Ý định: meeting Thời gian: [{"hour":15,"minute":0,"period":"chiều","raw":"lúc 15h chiều"}] Địa điểm: văn phòng Ghi chú: Không có	Ý định: meeting Thời gian: [{"hour":15,"minute":0,"period":"chiều","raw":"lúc 15h chiều"}] Địa điểm: văn phòng Ghi chú: Không có
Hẹn tôi học vẽ tranh	Ý định: study Thời gian: [{"hour":16,"minute":0,"peri	Ý định: study Thời gian: [{"hour":16,"minute":0,"per

lúc 16h chiều nay ở trung tâm mỹ thuật	od":"chiều","raw":"lúc 16h chiều"}] Địa điểm: trung tâm mỹ thuật Ghi chú: Không có	iod":"chiều","raw":"lúc 16h chiều"}] Địa điểm: trung tâm mỹ thuật Ghi chú: Không có
Hẹn tôi đi xem phim hoạt hình lúc 19h tối nay ở rạp Galaxy	Ý định: entertainment Thời gian: [{"hour":19,"minute":0,"period":"tối","raw":"lúc 19h tối"}] Địa điểm: rạp Galaxy Ghi chú: Không có	Ý định: entertainment Thời gian: [{"hour":19,"minute":0,"period":"tối","raw":"lúc 19h tối"}] Địa điểm: rạp Galaxy Ghi chú: Không có
Hẹn tôi học tiếng Anh lúc 17h chiều nay ở trung tâm ngoại ngữ	Ý định: study Thời gian: [{"hour":17,"minute":0,"period":"chiều","raw":"lúc 17h chiều"}] Địa điểm: trung tâm ngoại ngữ Ghi chú: Không có	Ý định: study Thời gian: [{"hour":17,"minute":0,"period":"chiều","raw":"lúc 17h chiều"}] Địa điểm: trung tâm ngoại ngữ Ghi chú: Không có